



Introduction to Makefiles

Michelle Shaw

ACEnet

May 2010 revision

Ross Dickson, Dalhousie Univ.



Why use make?

- Quick re-build of multi-step compile
- Build 3rd-party code
- Document the build procedure
- Portability to different platforms, compilers



Rebuilding

- Imagine a set of source files – `file1.c`, `file2.c`, ..., `file5.c` to build `program.x`
 - Six steps to build:

```
pgcc -c file1.c
pgcc -c file2.c ...
pgcc -o program.x file1.o file2.o ... file5.o
```
- Each time you update the source, you need to repeat at least one **compile** and one **link** step
- Use **make** to automate this



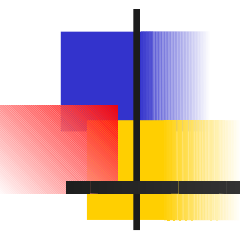
Building 3rd-party code

- Many linux packages install via “configure; make; make install” --- but sometimes it fails!
- Some packages say “Edit the Makefile to suit your local conditions”
- You need to understand Makefiles to do this



What is make?

- Consists of the command *make* and one or more user-defined files called Makefiles
- Makefile describes how to build one or more binaries
- Standard across many unix and linux systems
- This talk based upon GNU make



Makefile components



Rules

- A *Makefile* contains all the information needed to (re)build a package
 - Default name is “Makefile”, but can be anything
- Contains rules of this pattern:

```
target: prereq1 prereq2 ...  
    command1  
    command2  
    ...
```



Simple Makefile

```
editor : main.o kbd.o command.o display.o
        cc main.o kbd.o command.o display.o -o editor
```

```
main.o : main.c defs.h
```

```
        cc -c main.c
```

```
kbd.o : kbd.c defs.h command.h
```

```
        cc -c kbd.c
```

```
command.o : command.c defs.h command.h
```

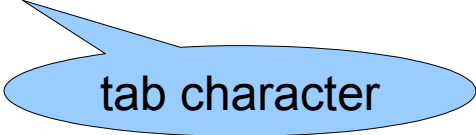
```
        cc -c command.c
```

```
display.o : display.c defs.h buffer.h
```

```
        cc -c display.c
```

```
clean :
```

```
        rm editor main.o kbd.o command.o display.o
```



tab character



How make works

- `make foo` means *foo* is the *target*
 - If none given, builds first target in Makefile
- Examines prerequisites for that target
- If there are rules for the prerequisites – *subtargets* – processes them first
- If the prerequisite file exists, checks the time stamp to see if it has changed more recently than the target



Makefiles – more pieces

- Variable declarations:
 - Define compilers, flags, linker, etc.
- Directives:
 - Done while make is processing the Makefile
 - Example: *include* statement
- Comments:
 - Useful for documentation
 - Begin with #



Makefile with variables

```
CC      = gcc
CFLAGS  = -fast
LD       = gcc
LDFLAGS = -lm -lc
```

```
C_OBJS = setparams.o read_ene.o read_proj_v2.o gen_xas_cpmd.o \
        rd_demon_inp.o linebroaden.o test_setparams.o
```

```
gen_xas: $(C_OBJS)
        $(LD) -o gen_xas $(LDFLAGS) $(C_OBJS)
```

```
.c.o:
        $(CC) $(CFLAGS) -c $*.c
```

```
clean:
        rm *.o gen_xas
```

line
continuation

“Old-fashioned
suffix rule”



Variables

- Declaration syntax:
`VARIABLE = value`
- Variables are referenced as `$(VAR)`, *e.g.:*
`$(CC) $(CFLAGS) -c $*.c`
- The parentheses are required
 - Without them *make* would interpret `$VAR` as `$V` followed by the literal “AR”
 - Single-letter variables not recommended!



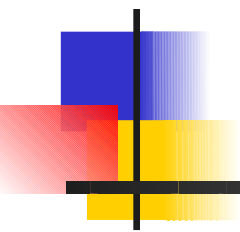
Variables

- Some variables have default values
 - list them with *make -p*
- Examples:
 - `CC = cc`
 - `FC = f77`
- Some variables are **automatic** :
 - `$@` file name of the target
 - `$?` list of prerequisites newer than the target
 - `$*` target name less any suffix, *e.g.* `ab.o` → `ab`
 - see manual for more



Rules miscellany

- Commands MUST start with tab character
- Target without a corresponding file *always* executes command
- Rules which are not depended on by other rules must be called explicitly
 - Common example: “clean” to clean out object files, Fortran module files, *etc.*



More complex Makefiles



Implicit rules

- Many *implicit* rules are built in to **make**
 - *e.g.* “use \$(CC) to turn *.c into *.o”
 - If **ab.o** is needed, make will look for **ab.c** and apply implicit rule
 - 3rd-party Makefiles may depend on these
 - List them with **make -p**



Pattern rules and suffix rules

- Implicit rules can be defined using GNU “pattern rule” syntax, or older “suffix rule” syntax

- Example pattern rule:

```
%.o : %.c
```

```
$(CC) -c $(CFLAGS) $(CPPFLAGS) $< -o $@
```

- Example suffix rule:

```
.c.o:
```

```
$(CC) $(CFLAGS) -c $*.c
```



Multiple Makefiles

- A large project may have Makefiles spread across several source directories
- Variables common to all Makefiles can be put in a global include file
 - *e.g.* a common compiler and flags
- Include it using:
`include $TOPDIR/make.inc`



Includes in Makefiles

- Using the previous example:
 - The make.inc file:

CC = gcc

CFLAGS =

LD = gcc

LDFLAGS = -lm -lc



Includes in Makefiles

- The rest of the Makefile:

```
include ./make.inc
```

```
C_OBJS = setparams.o read_ene.o read_proj_v2.o\  
        gen_xas_cpmd.o rd_demon_inp.o \  
        linebroaden.o test_setparams.o
```

```
all: $(C_OBJS)  
      $(LD) -o gen_xas $(LDFLAGS) $(C_OBJS)
```

```
.c.o:  
      $(CC) $(CFLAGS) -c $*.c
```

```
clean:  
      rm *.o gen_xas
```



A more complex example

- One can also use Makefiles to build separate packages and coordinate their builds with a high-level Makefile
 - Examples: Quantum-ESPRESSO, Elmer-FEM
- Especially handy when the parts of the package to build are independent of one another
- Example: set of source files to build/update as a set and grouped into directories by analysis type



A more complex example

- In root dir:
 - Makefile to control the calls to the Makefiles in the subdirectories
 - make.inc to set global variables
 - Five directories, each with their own Makefiles
 - To see options, type "make"
 - Choose from the list of options which codes to build



A more complex example

- Examples may be had from
glooscap:/var/tmp/make_ex2.tar.gz,
- Instructions are in
glooscap:/var/tmp/README_bldex,
and READMEs in extracted
subdirectories
- Build the example, then play around
with the options in make.inc



Querying make

- `make --dry-run foo`
 - Prints what `make` would do to build `foo`, but doesn't actually *do* it
- `make -p | less`
 - Shows default variables
 - Shows implicit rules



Some final notes

- Make can do much more but for most of what you will do this is sufficient
- Building Makefiles is part of the process of automating your tasks
- Learn by examining other Makefiles
- Learn by writing your own



More Information

- GNU make manual
 - <http://www.gnu.org/software/make/manual/>
- Google “make tutorial”
- ACEnet support: support@ace-net.ca
- Your local CRC